Derivative (Curvature) Regularization for ODE Model Fits

A step-by-step guide to the code and the math

Reference: `scripts/derivative_constraint_relay.R`
(machinery shared verbatim with `scripts/derivative_constraint.R`)

This note explains, from first principles, what `curvatureConstraint()` does and why it is built the way it is. It assumes you know a little R, but nothing about ODE sensitivity analysis, finite differences, or optimization theory – every piece of math is derived, not just stated.

Contents

1 The problem, in plain terms **1**

2 Minimal background **1**
- 2.1 States, observables, parameters 1
- 2.2 The objective, and why it needs a gradient and a Hessian 2
- 2.3 Sensitivities: $\partial Y/\partial\theta$, computed exactly 2

3 What "curvature" means here **2**

4 Approximating $\ddot{Y}$: finite differences from scratch **3**

5 From "the trajectory is rough" to "a number to minimize" **4**

6 Different scales: self-normalizing weights **4**

7 One interpretable knob: `rel_strength` **4**

8 The penalty function, assembled **5**

9 The gradient: exact, not finite-differenced **5**

10 The Hessian: a Gauss–Newton approximation **6**

11 Wiring it into dMod: a wrapper, not a +-term **7**

12 Getting states "for free": `attach.input` **7**

13 The dense-grid trick **7**

14 The demonstration model, and why it needs regularizing **8**
14.1 A feedforward relay – structurally incapable of oscillating 8
14.2 Practical non-identifiability, engineered by observation design 8
14.3 A real bug that is worth understanding: signal-to-noise 8
14.4 What actually happens without regularization 9

15 Checking against ground truth (only possible because this is simulated) **9**

16 The strength sweep: reading the tradeoff curve **9**

17 Guide to the saved plots **10**

18 Summary: the shape of the whole construction **10**

1 The problem, in plain terms

A dynamic model describes how a set of quantities ("states") change over time via a differential equation $\dot{X} = f(X, \theta)$, where θ is a vector of unknown parameters (rate constants, initial fractions, ...). Fitting the model means choosing θ so that the model's predictions $Y(t) = g(X(t))$ – the *observable* quantities, some function of the states – match measured data as closely as possible.

The trouble: when the data is sparse, noisy, or only covers *some* of the states, many different θ can fit the data almost equally well. This is called **practical non-identifiability** (as opposed to *structural* non-identifiability, which is a property of the model equations alone, true even with infinite perfect data – practical non-identifiability is a property of what you actually measured). Among all the equally-good-fitting θ's, nothing in the data rules out ones that produce biologically implausible trajectories: sharp jumps, ringing, wild excursions in the parts of the model that were never directly observed. A plain maximum-likelihood fit has no way to prefer the plausible member of that family over an implausible one – it only sees the likelihood, and by construction they all score about the same.

The fix explored here: add an extra term to the fitting objective that specifically penalizes *sharp corners in time* (formally: high second time-derivative, i.e. curvature), so the optimizer is nudged toward smoother, more plausible reconstructions – without needing to know in advance which specific parameter is the culprit.

2 Minimal background

2.1 States, observables, parameters

A state trajectory $X(t;\theta)$ solves $\dot{X} = f(X, \theta)$ from some initial condition. An observable is $Y(t;\theta) = g(X(t;\theta))$ – in the relay model, e.g. `obsTF` = `scale_TF` $\cdot$ `TFPnuc`, a scalar multiple of one state. Parameters are almost always fit on a log-scale (i.e. $\theta = \log(\text{rate constant})$) so that the optimizer's unconstrained search automatically respects that a rate constant must stay positive.

2.2 The objective, and why it needs a gradient and a Hessian

Fitting means minimizing an objective $\text{obj}(\theta)$ – essentially a (noise-weighted) sum of squared differences between model and data. Almost every practical optimizer needs, in addition to the

value obj(θ) itself:

- the **gradient** $\nabla_\theta \,\mathrm{obj}$ – which direction in parameter space improves the fit fastest;
- the **Hessian** $\nabla^2_\theta \,\mathrm{obj}$ – how much the objective curves in parameter space, used to size a good step.

A warning about the word "curvature": this document uses *curvature* in two completely different senses, and they must not be confused:

1. **Curvature in time** – how sharply a trajectory $Y(t)$ bends as t increases. This is what the new penalty targets, and it is a property of the *predicted trajectory*, nothing to do with θ directly.
2. **Curvature in parameter space** – the Hessian $\nabla^2_\theta \,\mathrm{obj}$, how the *objective* bends as you wiggle θ. This is what the optimizer needs to take good steps.

Confusingly, both matter here: we build a penalty on time-curvature (§3-8), and then we still need that penalty's own gradient/Hessian *in parameter space* (§9-10) so the optimizer can act on it.

2.3 Sensitivities: $\partial Y/\partial\theta$, computed exactly

To get an exact gradient of anything built from $Y(t;\theta)$, you need $\partial Y(t;\theta)/\partial\theta$ – how much the prediction at time t would change if you nudged parameter θ_k. dMod does not approximate this by finite-differencing θ (nudge θ_k, re-solve the ODE, see what changed); instead it integrates extra "sensitivity" equations *alongside* the model equations, giving the exact derivative at every time point as a direct by-product of the same ODE solve. You can treat this as a black box for the rest of this document: whenever a prediction is computed, an array

$$S(t) \;=\; \frac{\partial Y(t;\theta)}{\partial\theta} \quad \text{(shape [time, quantity, } \theta\text{])}$$

comes along with it, attached as `attr(prediction, "deriv")`, exactly and at no extra approximation cost.

3 What "curvature" means here

Think of $Y(t)$ as a position. Its first derivative $\dot{Y}(t)$ is a velocity – how fast it's changing. Its second derivative $\ddot{Y}(t)$ is an acceleration – how fast the *velocity* is changing, i.e. how sharply the curve is bending.

- A straight line has zero curvature everywhere, even if it is moving fast (large $\dot{Y}$, but constant slope means $\ddot{Y} = 0$).
- A smooth, gently-curving rise-and-fall (like a bell curve) has small, smoothly-varying curvature.
- A sharp corner – a kink where the slope changes abruptly – has, in the limit, curvature blowing up to infinity right at the kink.

This is exactly why the *second* derivative, not the first, is the right quantity to penalize (as discussed when this was first proposed): penalizing $\dot{Y}$ directly would also punish anything that moves fast, including genuinely fast-but-smooth dynamics or oscillation. Penalizing $\ddot{Y}$ specifically targets *corners*, and leaves fast-but-smooth motion alone.

4 Approximating $\ddot{Y}$: finite differences from scratch

We only ever have Y at a discrete set of times, so $\ddot{Y}(t)$ must be *approximated* from nearby values. The derivation, from Taylor's theorem:

$$Y(t+h) = Y(t) + h\,\dot{Y}(t) + \tfrac{h^2}{2}\,\ddot{Y}(t) + \tfrac{h^3}{6}\,\dddot{Y}(t) + O(h^4) \tag{1}$$

$$Y(t-h) = Y(t) - h\,\dot{Y}(t) + \tfrac{h^2}{2}\,\ddot{Y}(t) - \tfrac{h^3}{6}\,\dddot{Y}(t) + O(h^4) \tag{2}$$

Adding these two lines, the odd-order terms ($\dot{Y}$, $\dddot{Y}$) cancel exactly:

$$Y(t+h) + Y(t-h) \;=\; 2\,Y(t) + h^2\,\ddot{Y}(t) + O(h^4) \tag{3}$$

Rearranging for $\ddot{Y}(t)$:

$$\boxed{\ddot{Y}(t) \;\approx\; \frac{Y(t+h) - 2\,Y(t) + Y(t-h)}{h^2}}, \qquad \text{error } O(h^2). \tag{4}$$

This is the *central second difference* – and it is exactly `secondDiffTime()` (derivative_constraint_relay.R:255-

```
secondDiffTime <- function(arr, h) {
  n <- dim(arr)[1]
  lo <- 1:(n - 2); mid <- 2:(n - 1); hi <- 3:n
  if (length(dim(arr)) == 2) {
    (arr[hi, , drop = FALSE] - 2 * arr[mid, , drop = FALSE] + arr[lo, , drop = FALSE]) /
        h^2
  } else {
    (arr[hi, , , drop = FALSE] - 2 * arr[mid, , , drop = FALSE] + arr[lo, , , drop =
        FALSE]) / h^2
  }
}
```

`lo`, `mid`, `hi` are exactly $t-h$, t, $t+h$ in Equation (4), shifted by one index each. Because the formula needs a point on *both sides*, it can only be evaluated at *interior* grid points – the first and last time point have no 'before'/'after' neighbor, so the result array always has 2 fewer rows than the input (`n-2` instead of `n`). The function also handles a 3-dimensional array (`[time, quantity, θ]`) the same way, differencing along the time axis only – this is what makes it reusable for the sensitivity array too (§9).

This formula assumes the grid spacing h is the *same* everywhere. That is why `curvatureConstraint()` checks it explicitly:

```
stopifnot("curvatureConstraint() assumes a uniform time grid" =
          max(abs(hs - hs[1])) < 1e-8 * hs[1])
```

and why the fitting grid is deliberately built as an evenly-spaced sequence, `seq(0, 80, by = 0.1)` (§13).

5 From "the trajectory is rough" to "a number to minimize"

Equation (4) gives one curvature value per quantity j (a state or observable) per interior time point t_i:

$$\mathrm{curv}_j(t_i;\theta) \;=\; \frac{Y_j(t_{i+1};\theta) - 2\,Y_j(t_i;\theta) + Y_j(t_{i-1};\theta)}{h^2}.$$

To turn "how rough is quantity j's whole trajectory" into a single number, average its *squared* curvature over all interior time points (squared, not absolute value, for the same reason least-squares fitting uses squared residuals rather than absolute residuals: x^2 is smooth everywhere, including at $x = 0$, so the whole penalty stays differentiable):

$$\overline{\mathrm{curv}_j^2}(\theta) \;=\; \frac{1}{n_t}\sum_i \mathrm{curv}_j(t_i;\theta)^2, \qquad n_t = \text{number of interior time points.}$$

6 Different scales: self-normalizing weights

Different quantities live on very different numeric scales – a fast receptor pulse decaying from 0.8 to near 0, versus a slowly accumulating nuclear pool – and their *natural* curvature differs accordingly. Summing raw $\overline{\mathrm{curv}_j^2}$ across quantities would let whichever one happens to have the biggest numbers dominate the penalty, for reasons that have nothing to do with whether it is being fit *implausibly*.

The fix: give every quantity its own reference scale, measured once, at a reference fit called `best` (an ordinary, unregularized fit):

$$\mathrm{ref}_j \;=\; \overline{\mathrm{curv}_j^2}(\texttt{best}), \qquad \mathrm{weight}_j \;=\; \frac{1}{\mathrm{ref}_j + \varepsilon}.$$

```
ref <- colMeans(curv_best^2)
eps_curv <- eps_curv_frac * max(ref)
weight <- 1 / (ref + eps_curv)
```

Why this works: evaluated *at the reference point itself*, $\mathrm{weight}_j \cdot \overline{\mathrm{curv}_j^2}(\texttt{best}) = \mathrm{ref}_j/(\mathrm{ref}_j + \varepsilon) \approx 1$ for every quantity j (as long as $\mathrm{ref}_j \gg \varepsilon$) – regardless of that quantity's raw numeric scale. So every state and observable starts out contributing *about the same amount* to the total penalty; only *deviations* from the reference curvature (getting rougher, in either direction) actually move the penalty.

$\varepsilon = \texttt{eps_curv_frac} \times \max_j(\mathrm{ref}_j)$ is a tiny floor (a *proportional* one, not a fixed absolute number, so it scales with whatever units the model happens to use) that prevents a division blow-up for a quantity that happens to be almost perfectly flat at `best` (where $\mathrm{ref}_j \approx 0$).

7 One interpretable knob: `rel_strength`

We still need an overall scale for how hard to enforce smoothness. Rather than an arbitrary absolute number, it is defined as a *fraction of the model's own likelihood value at the reference fit*:

$$\mathrm{strength_curv} \;=\; \mathrm{rel_strength} \times \big|\mathrm{obj}(\texttt{best})\big|.$$

```
strength_curv <- rel_strength * abs(out_best$value)
```

Why `abs()` – a real bug, worth understanding. The fitting objective's value need not be positive: it is a full Gaussian negative log-likelihood, and for small measurement noise σ its normalization term ($\propto \log\sigma^2$) can dominate and make the *total* value negative (it does, here: $\mathrm{obj}(\texttt{best}) \approx -54.7$). Without `abs()`, `strength_curv` would inherit that sign and turn *negative* – and a negative-strength penalty *rewards* roughness instead of discouraging it. This was caught

empirically while building this script (Section 14 revisits the general lesson it points to), which is exactly the kind of subtlety that is easy to miss on paper and cheap to catch by testing.

Why this choice of normalization is interpretable. Averaging (rather than summing) over both quantities and time points – $C = \text{strength_curv}/(n_{\text{cols}} \cdot n_t)$ – means that, evaluated at `best` itself,

$$R(\texttt{best}) \;=\; C \sum_j \text{weight}_j \sum_i \text{curv}_j(t_i; \texttt{best})^2 \;\approx\; \frac{\text{strength_curv}}{n_{\text{cols}}} \sum_j 1 \;=\; \text{strength_curv},$$

using the ≈ 1 result from §6. So `rel_strength = 0.1` means, concretely: *at the reference fit, the smoothness penalty is worth about 10% of the data-fit term* – a directly interpretable statement, rather than an arbitrary number with no intrinsic scale.

8 The penalty function, assembled

Putting §5-7 together, the full penalty is

$$R(\theta) \;=\; C \sum_{j=1}^{n_{\text{cols}}} \text{weight}_j \sum_{i=1}^{n_t} \text{curv}_j(t_i; \theta)^2, \qquad C = \frac{\text{strength_curv}}{n_{\text{cols}}\, n_t}. \tag{5}$$

```
pen_value <- C * sum(vapply(seq_len(n_cols), function(j)
  weight[j] * sum(curv[, j]^2), numeric(1)))
```

Here `curv` is `secondDiffTime(Y, hs)` applied to the trajectory *at the current* θ being tried by the optimizer – not at `best`. `best` only ever supplies the fixed reference weights.

9 The gradient: exact, not finite-differenced

The optimizer needs $\partial R/\partial\theta_k$. By the ordinary chain rule applied to Equation (5):

$$\frac{\partial R}{\partial \theta_k} \;=\; 2C \sum_j \text{weight}_j \sum_i \text{curv}_j(t_i; \theta)\, \frac{\partial\, \text{curv}_j(t_i; \theta)}{\partial \theta_k}. \tag{6}$$

Everything hinges on that last factor. Write out $\text{curv}_j(t_i;\theta) = [Y_j(t_{i+1};\theta) - 2Y_j(t_i;\theta) + Y_j(t_{i-1};\theta)]/h^2$: it is a *fixed linear combination* (with coefficients $1, -2, 1$, divided by h^2 – none of which depend on θ) of Y_j evaluated at three times. Because differentiation is linear – $\partial(aU + bV)/\partial\theta_k = a\,\partial U/\partial\theta_k + b\,\partial V/\partial\theta_k$ for constants a, b – differentiating this fixed combination with respect to θ_k just applies the *same* combination to the derivatives:

$$\frac{\partial\, \text{curv}_j(t_i; \theta)}{\partial \theta_k} \;=\; \frac{S_j(t_{i+1};\theta)_k - 2\, S_j(t_i;\theta)_k + S_j(t_{i-1};\theta)_k}{h^2}, \qquad S_j(t;\theta)_k \equiv \frac{\partial Y_j(t;\theta)}{\partial \theta_k}. \tag{7}$$

This is *the same finite-difference operator*, applied not to Y but to the sensitivity S – because the time grid h never depends on θ, differencing in time and differentiating in θ **commute exactly**. There is no approximation introduced by this step beyond the one already made in §4 for the trajectory itself. This is the central trick of the whole construction, and it is why the code can reuse `secondDiffTime()` completely unchanged:

```
dY <- attr(pred, "deriv")[, cols, , drop = FALSE] # S(t), exact, from dMod
dcurv <- secondDiffTime(dY, hs) # exactly Eq. (7)
```

Substituting Equation (7) into Equation (6) and writing it as a matrix product (summing over i for every k at once, instead of looping):

```
for (j in seq_len(n_cols)) {
  Jc <- dcurv[, j, ] # matrix: rows = time, cols = theta
  dim(Jc) <- c(dim(dcurv)[1], length(theta))
  pen_gradient <- pen_gradient + 2 * C * weight[j] * as.vector(curv[, j] %*% Jc)
  ...
}
```

`curv[, j] %*% Jc` computes, in one matrix-vector product, $\sum_i \mathrm{curv}_j(t_i) \cdot \mathrm{Jc}_{i,k}$ for every θ_k simultaneously – exactly the sum in Equation (6). The θ-derivative of the whole penalty is therefore **exact given the time discretization** – θ itself is never finite-differenced, only time is (and only once, in §4).

10 The Hessian: a Gauss–Newton approximation

The optimizer also wants $\partial^2 R/\partial\theta_k\partial\theta_l$. Differentiating Equation (6) once more with respect to θ_l (product rule on the two θ-dependent factors) gives

$$\frac{\partial^2 R}{\partial\theta_k\partial\theta_l} = 2C \sum_j w_j \sum_i \left[\frac{\partial\, \mathrm{curv}_j(t_i)}{\partial\theta_k} \frac{\partial\, \mathrm{curv}_j(t_i)}{\partial\theta_l} \;+\; \mathrm{curv}_j(t_i)\, \frac{\partial^2\, \mathrm{curv}_j(t_i)}{\partial\theta_k\partial\theta_l} \right].$$

The second term needs *second*-order sensitivities ($\partial^2 Y/\partial\theta_k\partial\theta_l$) – more expensive, and not always available. The standard fix (the same one `normL2()` itself uses by default, and the same one behind the Fisher-information construction discussed previously for the FIM/L2 regularization) is the **Gauss–Newton approximation**: drop the second term entirely and keep only the first, which is a sum of *squares* of first derivatives:

$$\frac{\partial^2 R}{\partial\theta_k\partial\theta_l} \;\approx\; 2C \sum_j w_j \sum_i \frac{\partial\, \mathrm{curv}_j(t_i)}{\partial\theta_k} \frac{\partial\, \mathrm{curv}_j(t_i)}{\partial\theta_l}. \tag{8}$$

```
pen_hessian <- pen_hessian + 2 * C * weight[j] * (t(Jc) %*% Jc)
```

`t(Jc) %*% Jc` computes exactly the $(\theta\times\theta)$ matrix whose (k,l) entry is $\sum_i \mathrm{Jc}_{i,k}\,\mathrm{Jc}_{i,l}$ – Equation (8), for every pair (k,l) at once. Two reasons this approximation is preferred, not just tolerated: (1) it is **guaranteed positive semi-definite** (any matrix of the form $J^\top J$ is), which keeps the optimizer numerically well-behaved, whereas the dropped exact term has no such guarantee; (2) it avoids computing second-order sensitivities at all, which is both cheaper and simpler.

11 Wiring it into dMod: a wrapper, not a +-term

dMod's usual idiom composes penalties with `+` (`obj + constraintL2(...)`), and each summand independently calls the model. That would be wasteful here: the penalty needs the model's *full*

predicted trajectory, so a separately-composed penalty term would re-solve the ODE a second time on every single objective evaluation.

Instead, `curvatureConstraint()` builds a **wrapper**: it calls the base objective (`normL2`) *once*, and reads the prediction that call already computed and cached (`attr(base_out, "env")$prediction`) before adding the penalty on top:

```
wrapped <- function(..., fixed = NULL, deriv = TRUE, deriv2 = FALSE, env = NULL) {
  base_out <- base_obj(..., fixed = fixed, deriv = deriv, deriv2 = deriv2, env = env)
  pred <- attr(base_out, "env")$prediction[[condition]]
  ... # build pen_value / pen_gradient / pen_hessian from 'pred'
  combined <- objlist(value = base_out$value + pen_value, ...)
  attr(combined, "env") <- attr(base_out, "env")
  combined
}
class(wrapped) <- class(base_obj)
attr(wrapped, "conditions") <- attr(base_obj, "conditions")
attr(wrapped, "parameters") <- attr(base_obj, "parameters")
```

One ODE solve (with sensitivities) serves both the likelihood *and* the curvature penalty. The last three lines matter more than they look: without them, `wrapped` is just a plain R function, and dMod's own internals – specifically `mstrust()`, which calls a generic function `mname()` that dispatches on class – fail with "no applicable method for 'mname' applied to an object of class 'function'". Copying the base objective's class and attributes is what makes the wrapper behave, to the rest of dMod, exactly like an ordinary objective function.

12 Getting states "for free": `attach.input`

The penalty is meant to cover *every* state, not just the measured observable(s) – but only the observable was ever fit to data. Two observation functions are built from the same underlying model:

```
g <- Y(observables, x, ..., attach.input = FALSE) # observables only
g_full <- Y(observables, x, ..., attach.input = TRUE) # + every raw state
```

Internally, computing $Y(t) = g(X(t))$ always requires solving for the full state trajectory $X(t)$ first – the observable is a cheap algebraic function applied *afterward*. `attach.input` only controls whether that already-computed state trajectory (and its already-exact sensitivities) is kept in the output or discarded; it costs one array append, not a second ODE solve. `g` (states discarded) is used only to generate the simulated dataset, so the simulated "data" never accidentally includes a measurement of a state that was never meant to be observed; `g_full` (states kept) is used to build the fitting objective, so the curvature penalty has every state available.

13 The dense-grid trick

```
times_fit <- seq(0, 80, by = 0.1)
obj <- normL2(data, g_full * x * p, times = times_fit)
```

Fitting happens on a dense, *evenly-spaced* grid, even though the actual data (`times_data <- seq(0, 80, by = 8)`) is far sparser. This does not change the fitted likelihood at all: `normL2()`'s

internal scoring only reads the rows matching real data timestamps; every other row in the dense grid is simply extra, unused-by-the-likelihood output that the ODE solver was asked to also produce. The same expensive integration that computes the likelihood therefore produces, as an unavoidable side effect, exactly the dense, uniformly-spaced array the curvature penalty needs (§4's uniform-grid requirement) – at no extra integration cost.

14 The demonstration model, and why it needs regularizing

14.1 A feedforward relay – structurally incapable of oscillating

The model (`derivative_constraint_relay.R:105-132`) is a strict one-way relay: a decaying receptor pulse drives `S1`, which drives `S2`, which drives a two-step `S3` tier, which drives a two-step `S4` tier, which drives `TF`→`TFP`→`TFPnuc`. Every reaction only ever regulates something *downstream* of itself; nothing feeds back. A chain of one-way handoffs like this can only rise and fall once per stage – there is no mechanism available to create the ringing/overshoot that a feedback loop (like the MAPK cascade's inhibitory loop in `derivative_constraint.R`) can produce. This isolates the phenomenon of interest from oscillation entirely.

14.2 Practical non-identifiability, engineered by observation design

Only the very last quantity, `obsTF` = `scale_TF`×`TFPnuc`, is ever measured, and only every 8 time units, with noise. Every one of the other 16 states – in particular `S2`/`S2P`, four reaction steps upstream – is never observed. Mathematically, the model is still *structurally* identifiable (with infinite, perfect, fully-observed data, every parameter would be recoverable); but with *this* sparse, single-readout, noisy dataset, many different upstream trajectories can produce statistically indistinguishable downstream signals. That gap between "identifiable in principle" and "identifiable from this actual data" is exactly what *practical* non-identifiability means.

14.3 A real bug that is worth understanding: signal-to-noise

The first version of this model used `scale_TF = 1`, giving `obsTF` a true range of about 0–0.0009 – thirty times *smaller* than the assumed noise $\sigma = 0.03$. With noise that much bigger than the entire signal, the "data" is close to pure noise, and *any* reasonable fit looks about as good as any other – for the wrong reason (no real signal to speak of), not because of the intended mechanism (sparse sampling of a real signal). This was caught empirically by printing the true trajectory's actual range and comparing it to σ, and fixed by setting `scale_TF = 1000`, bringing `obsTF`'s peak to ≈ 0.92 – a sane few-percent relative noise level. The general lesson: whenever you fix a noise level by analogy to a different model, check that it is actually small relative to *this* signal's own dynamic range, or you are not testing what you think you are.

14.4 What actually happens without regularization

A 100-start multistart's single best-likelihood (converged) fit reconstructs `S2P` with roughness $\approx 8.6\times$ the true trajectory's own roughness, and RMSE ≈ 0.15 against the true trajectory – while paying almost nothing in likelihood for it. Several other upstream/interior quantities are worse still (e.g. `S3PP`: RMSE 0.95, roughness 102× truth). This is the mechanism working exactly as engineered: the optimizer is free to overfit noise into the *unobserved* parts of the model, because nothing in the sparse downstream data forbids it.

15 Checking against ground truth (only possible because this is simulated)

Because the data is simulated, the *true* trajectory is known – something you would never have with real data. `roughnessAll()` and `rmseVsTruth()` (derivative_constraint_relay.R:208-217) use it purely as an independent check, never fed into either fit:

```
roughnessAll <- function(pars) {
  pred <- (g_full * x * p)(times_fit, pars)[[1]][, cols_all, drop = FALSE]
  n <- nrow(pred)
  curv <- (pred[3:n,] - 2*pred[2:(n-1),] + pred[1:(n-2),]) / hstep^2 # Eq. (4)
  sqrt(colMeans(curv^2)) # RMS curvature
}
rmseVsTruth <- function(pars) {
  pred <- (g_full * x * p)(times_fit, pars)[[1]][, cols_all, drop = FALSE]
  sqrt(colMeans((pred - true_full[, cols_all, drop = FALSE])^2)) # standard RMSE
}
```

Applying the curvature penalty (`rel_strength = 0.1`) moves almost every quantity toward the truth on *both* metrics at once, at a likelihood cost of 0.004 (raw likelihood $-54.675 \to -54.671$) while the total curvature penalty itself drops from 5.42 to 0.07 ($\sim 73\times$) – e.g. `S3PP` RMSE $0.95 \to 0.004$ ($243\times$ better), `TFP` RMSE $0.049 \to 0.0002$ ($300\times$ better). `RecA`/`Rec` are the one exception, and it is a ratio artifact rather than a real failure: `RecA` is a pure single-exponential decay with near-zero true curvature, so even a tiny absolute roughness inflates the *ratio* against that near-zero reference.

16 The strength sweep: reading the tradeoff curve

`rel_strength = 0.1` was picked by hand. Part 11 of the script repeats the whole construction across rel_strength $\in \{0.01, 0.1, 1, 10, 100\}$ (a log-grid, reusing the already-computed 0.1 run rather than refitting it), and pools baseline + all five strengths into one combined waterfall (capped to the best 400 of 600 fits by value, using the same `topByValue()` helper introduced for fim_l2_constraintL2.R – see that companion note – so one badly-diverged fit can't stretch the y-axis and hide everything else).

The actual result traces out a textbook bias/cost tradeoff curve:

`rel_strength`	raw likelihood	curvature penalty	RMSE(`S2P`)	roughness(`S2P`)/truth
0.01	−54.675	0.011	0.0306	0.437
0.1	−54.671	0.074	0.0322	0.091
1	−54.616	0.533	0.0326	0.018
10	−54.262	4.139	0.0357	0.003
100	−45.970	30.661	0.0365	0.003

Raw likelihood is essentially flat from 0.01 through 10 (a change of 0.4, against a baseline around −55), then collapses sharply at 100 (−45.97) – a clear, numerically visible signature that the penalty has started to actively fight the data fit rather than just cleaning up unconstrained roughness. Meanwhile `S2P`'s RMSE against truth barely moves across the whole range (already fixed by even the weakest regularization), while its roughness keeps falling smoothly. The practical

reading: anywhere from 0.01 to 10 is a reasonable choice here; 100 is visibly over-regularized, and the likelihood column is what tells you so, not just visual inspection of a plot.

17 Guide to the saved plots

All plots live in `scripts/plots/relay_derivative_constraint/`.

- `..._baseline_waterfall.pdf`, `..._curvature_waterfall.pdf` – one multistart's sorted-by-value fits (§16's per-run waterfalls), unregularized vs. `rel_strength=0.1`.
- `..._observable.pdf` – truth / baseline / curvature-regularized, for the one thing actually fit to data (`obsTF`).
- `..._all_states.pdf` – the same three trajectories for *every* state – this is where the smoothing effect on the unobserved upstream tiers becomes visible directly.
- `..._str{0.01,1,10,100}_waterfall.pdf` – the individual per-strength waterfalls from the sweep.
- `..._strength_sweep_waterfall.pdf` – all six runs (baseline + 5 strengths) pooled into one waterfall, colored by group.
- `..._strength_sweep_observable.pdf`, `..._strength_sweep_all_states.pdf` – truth, baseline, and the best fit at every strength, overlaid.

18 Summary: the shape of the whole construction

1. Approximate $\ddot{Y}$ by a central second difference in time (Taylor series, §4) – the *only* approximation in the whole construction.
2. Average its square per quantity, self-normalize by each quantity's own reference curvature at a baseline fit (§6), scale the total by a fraction of the baseline likelihood (§7).
3. Get the exact parameter-gradient of that discretized penalty by applying the *same* fixed linear operator to the model's own exact sensitivities, since a fixed linear time-operator and $\partial/\partial\theta$ commute (§9) – no finite-differencing of θ, ever.
4. Approximate the Hessian by Gauss–Newton, for the same reasons `normL2()` itself does (guaranteed positive semi-definite, no second-order sensitivities needed) (§10).
5. Wrap, rather than `+`-compose, so one ODE solve serves both the likelihood and the penalty (§11); reuse the intermediate state trajectory that observation functions compute anyway (§12); fit on a dense grid that costs nothing extra because the likelihood only reads the sparse subset it actually needs (§13).